



АНАЛИЗ ЗАЩИЩЕННОСТИ ПО (НА ВЕБ)

ПРАКТИЧЕСКОЕ ЗАДАНИЕ: Анализ защищённости веб-приложений

Выполнил: Юрий Шамрай (MIFIIB/1-й поток)

ЦЕЛЬ ПРАКТИЧЕСКОГО ЗАДАНИЯ:

Овладение навыками анализа защищённости веб-приложений на примере OWASP Juice Shop, выявление и документирование уязвимостей из OWASP Top-10, анализ связанных с ними CWE, а также разработка рекомендаций для их устранения.

ЧТО НУЖНО СДЕЛАТЬ:

Проведите анализ защищённости приложения Juice Shop. В отчёте укажите уязвимости из OWASP Top-10; CWE, которые их вызывают, а также рекомендации по их устранению.

- Отчёт должен иметь следующую структуру:
 - ✓ Введение:
 - описание приложения;
 - ✓ Результаты статического анализа:
 - общие, можно без детализации;
 - ✓ Уязвимости из OWASP Top-10, обнаруженные в результате статического анализа:
 - не менее 5 штук;
 - ✓ Демонстрация эксплуатации трёх уязвимостей из OWASP Top-10:
 - скриншоты эксплуатации, проведённой с помощью инструмента Burp Suite;
 - ✓ Рекомендации по устранению к трём продемонстрированным уязвимостям:
 - можно взять основу с сайта MITRE ATT&CK под найденные CWE.

УСЛОВИЯ РЕАЛИЗАЦИИ:

- В качестве отчёта предоставьте следующее:
 - ✓ Скриншоты (минимум два) или выгрузка результатов статического сканирования в пункте 2.
 - ✓ Список уязвимостей из OWASP Top-10 с доказательствами в пункте 3 (в виде скриншотов, найденных анализатором уязвимостей; отдельно от остальных уязвимостей).
 - ✓ Скриншоты из Burp Suite в пункте 4, а также обязательно текстовое описание эксплуатации: что необходимо сделать, чтобы воспроизвести уязвимость.
 - ✓ Не обязательно выбирать три разные уязвимости. Вы можете выбрать три вариации одной и той же уязвимости.
 - ✓ Пришлите отчёт в формате PDF о проделанной работе.

ВВЕДЕНИЕ

Данная практическая работа направлена на анализ защищённости приложения "OWASP Juice Shop" с использованием методов статического анализа и тестирования на проникновение, а также, на повышение понимания уязвимостей в веб-приложениях и разработку навыков по их выявлению и устранению с целью обеспечения безопасности приложений.

Целью практической работы является:

1. Проведение анализа защищённости приложения Juice Shop, сосредоточившись на выявлении уязвимостей, соответствующих OWASP Top-10;
2. Определение общих результаты статического анализа приложения;
3. Идентифицировать и задокументировать минимум пять уязвимостей из OWASP Top-10, включая их классификацию CWE (Common Weakness Enumeration) и короткое описание их вызывающих факторов;
4. Продемонстрировать эксплуатацию трех из выявленных уязвимостей, используя инструмент Burp Suite, предоставив скриншоты эксплуатации и текстовое описание шагов для воспроизведения уязвимости;
5. Предоставить рекомендации по устранению трех продемонстрированных уязвимостей, для улучшения общей безопасности приложения Juice Shop.

Выполнение данного практического задания потребует ряда шагов и действий для проведения анализа защищённости приложения Juice Shop. Ниже приведено подробное описание о том, что нам предстоит сделать:

1. Загрузка и подготовка Juice Shop:

- Загружаем и устанавливаем приложение Juice Shop, доступное на GitHub;
- Запуск и проверка его работоспособности.

2. Статический анализ:

- Использование инструментов для статического анализа кода (Semgrep, Burp Suite) для обнаружения общих уязвимостей и слабых мест в коде приложения;
- Документирование общих результатов статического анализа.

3. Выявление уязвимостей OWASP Top-10:

- Исследовать приложение, фокусируясь на уязвимостях, входящих в список OWASP Top-10 (инъекции, брешь в аутентификации);

4. Демонстрация эксплуатации уязвимостей:

- Выбрать три из выявленных уязвимостей и продемонстрировать их эксплуатацию с использованием инструмента Burp Suite;
- Создать скриншоты, иллюстрирующие каждый этап эксплуатации с подробным описанием шагов, необходимых для воспроизведения уязвимости.

5. Рекомендации по устранению:

- Для каждой продемонстрированной уязвимости предложить рекомендации по её устранению;

6. Оформление отчёта:

- Создать структурированный отчёт в формате PDF согласно указанным требованиям, включая скриншоты, описания и рекомендации;
- Обязательно указать информацию о приложении Juice Shop, его цели и контексте выполнения задания.

Это практическое задание поможет нам лучше понять уязвимости в веб-приложениях и развить навыки их выявления и устранения.

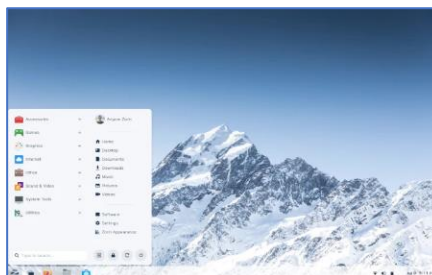
Практическое задание мы будем выполнять на виртуальной машине:

В качестве гипервизора используем [VirtualBox 7.0.10](#)



Сборка операционной системы Linux:

- [Zorin OS 16.3 Core](#)



со следующими параметрами:

1. ZORIN-VM (OWASP Juice Shop)

- OS: Zorin OS 16.3 (Core)
- CPU: 1CPU x 4vCORES
- RAM: 4 GB
- HDD: 80 GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
prospero@zorin-vm:~$ cat /etc/os-release
NAME="Zorin OS"
VERSION="16.3"
ID=zorin
```

OWASP JUICE SHOP

Это небольшое веб-приложение, которое разработано как набор уязвимостей с открытым исходным кодом с целью обучения и тренировки специалистов по информационной безопасности, а также для проведения тестирования на проникновение. Это приложение создано и поддерживается сообществом Open Web Application Security Project (OWASP) и является одним из наиболее популярных проектов, предназначенных для обучения и демонстрации уязвимостей в веб-приложениях.

1. **Цель и назначение:** Juice Shop — это обучение и практика анализа уязвимостей веб-приложений. Он предоставляет безопасное окружение для тестирования различных видов атак, таких как инъекции, переполнение буфера, кросс-сайтовый скриптинг (XSS), аутентификационные и авторизационные уязвимости и многие другие. Он также используется в области обучения и сертификации специалистов по информационной безопасности;
2. **Технологии и стек:** Juice Shop построен с использованием современных веб-технологий и стека, включая Node.js для серверной части, Angular для фронтенда, и базу данных SQLite. Это делает его легко доступным и запускаемым на большинстве платформ;
3. **Уровень сложности:** Juice Shop предлагает уровень сложности, который может варьироваться от начинающего до продвинутого. Пользователи могут выбирать уровень сложности в зависимости от своего опыта и навыков;

4. **Задачи и функциональность:** Приложение включает в себя разнообразные задачи и функциональность, связанные с безопасностью, такие как нахождение уязвимостей, решение криптографических головоломок, анализ отчетов о безопасности и многое другое;
5. **Интерактивность и обратная связь:** Juice Shop предоставляет интерактивную среду для обучения и тестирования. Пользователи могут получать мгновенную обратную связь о своих действиях и уязвимостях, что помогает им учиться на практике;
6. **Документация и сообщество:** Juice Shop имеет обширную документацию, включая инструкции по установке и использованию. Сообщество OWASP активно поддерживает проект и предоставляет ресурсы для помощи пользователям;
7. **Исходный код с открытым доступом:** Исходный код OWASP Juice Shop полностью открыт и доступен на платформе GitHub, что позволяет пользователям изучать его, вносить вклад и создавать собственные версии для обучения и тестирования.

В целом, OWASP Juice Shop - это мощный инструмент для обучения и тренировки в области информационной безопасности, который обеспечивает практический опыт в обнаружении и устранении уязвимостей в веб-приложениях, что является важным аспектом в обеспечении безопасности современных веб-проектов.

Установка OWASP Juice Shop

Чтобы установить OWASP Juice Shop на Zorin OS, выполним следующие шаги:

Устанавливаем зависимости

Нам потребуются [node.js](#) [npm](#) [git](#)

в терминале выполним следующие команды для проверки версий:

\$ node -v

\$ npm -v

\$ git --version

```
prospero@zorin-vm:~$ node -v
Command 'node' not found, but can be installed with:
sudo apt install nodejs

prospero@zorin-vm:~$ npm -v
Command 'npm' not found, but can be installed with:
sudo apt install npm

prospero@zorin-vm:~$ git --version
Command 'git' not found, but can be installed with:
sudo apt install git
```

Видим, что необходимые пакеты отсутствуют, тогда выполним их установку:

Git:

\$ sudo apt install -y git

```
prospero@zorin-vm:~$ git --version
git version 2.25.1
```

Node.js, npm:

Нам потребуется скачать и распаковать архив с бинарными файлами Node.js:

Скачиваем архив с Node.js с официального сайта: <https://nodejs.org/en/download/current>

https://nodejs.org/en/download/current			
Windows installer (.msi)	32-bit	64-bit	ARM64
Windows Binary (.zip)	32-bit	64-bit	ARM64
macOS Installer (.pkg)	64-bit / ARM64		
macOS Binary (.tar.gz)	64-bit		ARM64
Linux Binaries (x64)	64-bit		

В терминале переходим в папку, куда скачали архив и распаковываем командой:

```
$ tar -xvf node-v20.6.1-linux-x64.tar.xz
```

```
prospero@zorin-vm:~$ cd Downloads/  
prospero@zorin-vm:~/Downloads$ ls  
node-v20.6.1-linux-x64.tar.xz  
prospero@zorin-vm:~/Downloads$ tar -xvf node-v20.6.1-linux-x64.tar.xz
```

Далее, переходим в распакованную папку Node.js и скопируем содержимое этой папки в системную папку, чтобы Node.js был доступен системно:

```
$ cd node-v20.6.1-linux-x64/
```

```
$ sudo cp -R * /usr/local/
```

```
prospero@zorin-vm:~/Downloads$ cd node-v20.6.1-linux-x64/  
prospero@zorin-vm:~/Downloads/node-v20.6.1-linux-x64$ sudo cp -R * /usr/local/
```

Также, можно обновить версию npm командой (опционально):

```
$ sudo npm install -g npm@10.1.0
```

Проверяем, что Node.js и npm (пакетный менеджер для Node.js) установлены корректно:

```
$ node -v
```

```
$ npm -v
```

```
prospero@zorin-vm:~$ node -v  
v20.6.1  
prospero@zorin-vm:~$ npm -v  
10.1.0
```

Следующим шагом клонируем репозиторий OWASP Juice Shop с GitHub в папку ~/Downloads:

```
$ cd
```

```
$ git clone https://github.com/juice-shop/juice-shop.git ~/Downloads/juice-shop --depth 1
```

```
prospero@zorin-vm:~/Downloads/node-v20.6.1-linux-x64$ cd  
prospero@zorin-vm:~$ git clone https://github.com/juice-shop/juice-shop.git ~/Downloads/juice-shop --depth 1  
Cloning into '/home/prospero/Downloads/juice-shop'...  
remote: Enumerating objects: 1199, done.  
remote: Counting objects: 100% (1199/1199), done.  
remote: Compressing objects: 100% (887/887), done.  
remote: Total 1199 (delta 349), reused 848 (delta 293), pack-reused 0  
Receiving objects: 100% (1199/1199), 44.77 MiB | 10.46 MiB/s, done.  
Resolving deltas: 100% (349/349), done.
```

“--depth 1” – опция означает, что будет клонирован только один последний коммит (без истории).

Переходим в каталог, который был создан в результате клонирования репозитория:

```
$ cd Downloads/juice-shop/
```

Запускаем установку зависимостей проекта командой:

\$ npm install # эта команда используется только перед первым запуском или при изменении исходного кода.

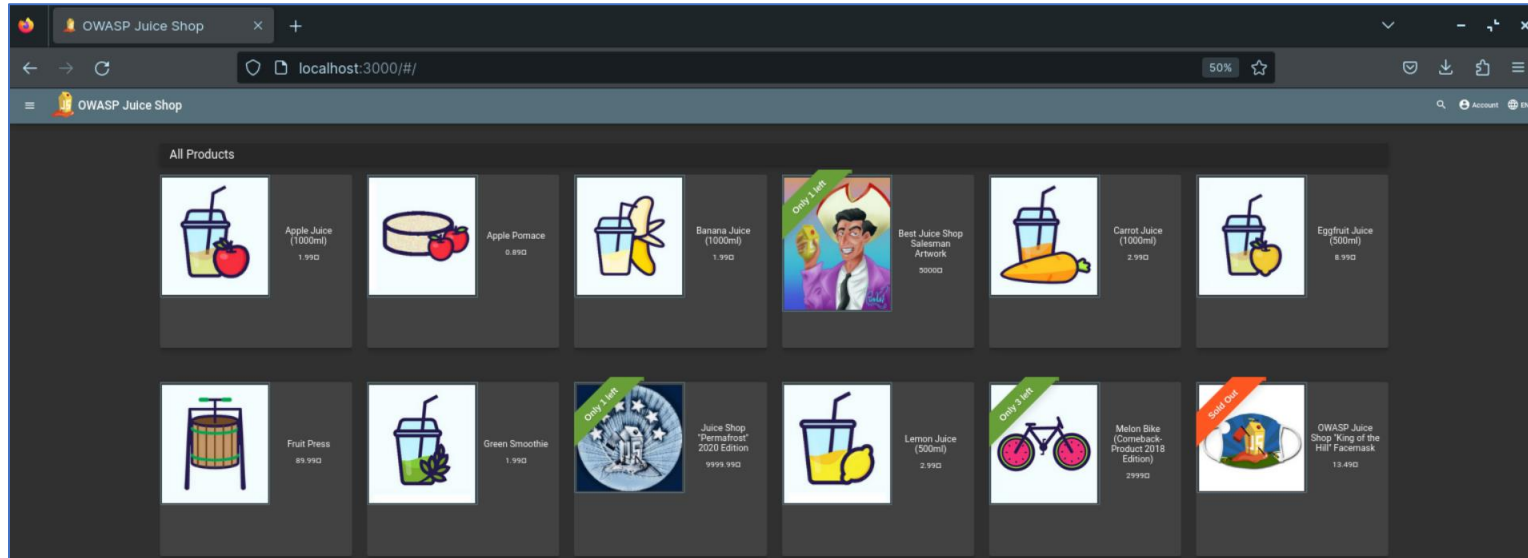
```
prospero@zorin-vm:~$ cd Downloads/juice-shop/  
prospero@zorin-vm:~/Downloads/juice-shop$ npm install  
( ) !: idealTree:juice-shop: sill idealTree buildDeps
```

После установки зависимостей запускаем OWASP Juice Shop командой:

```
$ npm start
```

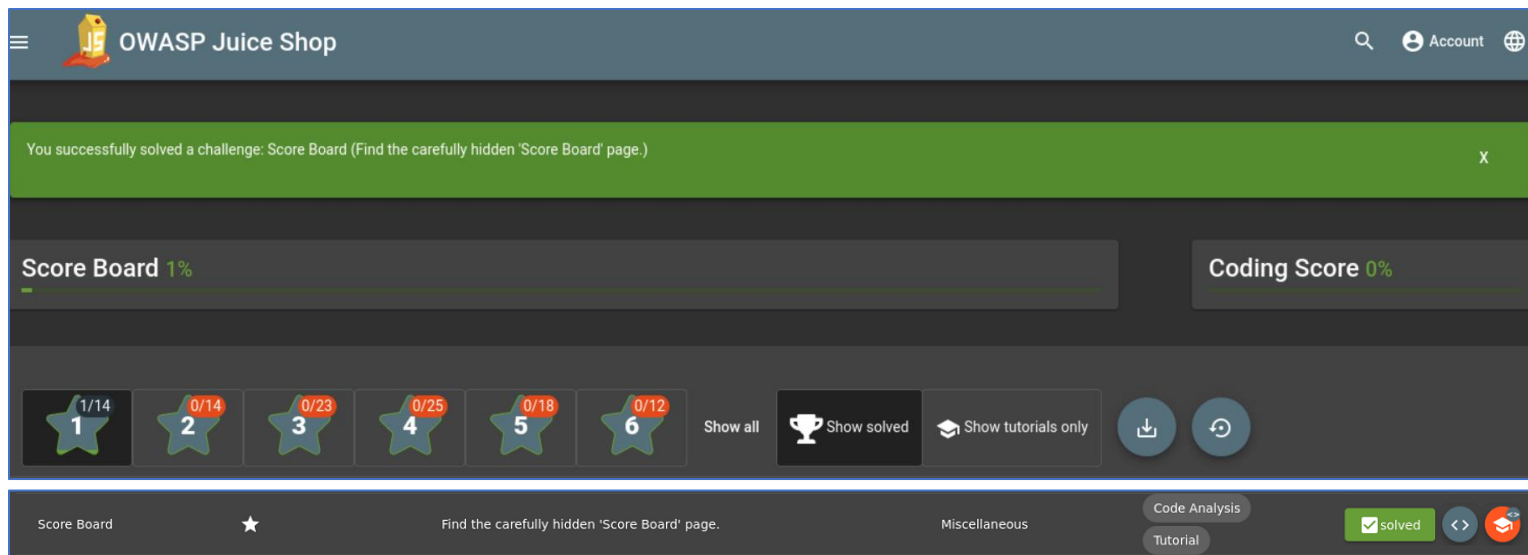
```
prospero@zorin-vm:~/Downloads/juice-shop$ npm start  
  
> juice-shop@15.1.0 start  
> node build/app  
  
info: All dependencies in ./package.json are satisfied (OK)  
info: Detected Node.js version v20.6.1 (OK)  
info: Detected OS linux (OK)  
info: Detected CPU x64 (OK)  
info: Configuration default validated (OK)  
info: Entity models 19 of 19 are initialized (OK)  
info: Required file server.js is present (OK)  
info: Required file index.html is present (OK)  
info: Required file styles.css is present (OK)  
info: Required file main.js is present (OK)  
info: Required file polyfills.js is present (OK)  
info: Required file runtime.js is present (OK)  
info: Required file vendor.js is present (OK)  
info: Port 3000 is available (OK)  
info: Chatbot training data botDefaultTrainingData.json validated (OK)  
info: Server listening on port 3000
```


Открываем веб-браузер и переходим по адресу <http://localhost:3000> для доступа к OWASP Juice Shop:



Также, мы можем выполнить самый первый «челендж» найти скрытую страницу “Score Board”:

В адресной строке добавляем `/score-board`:

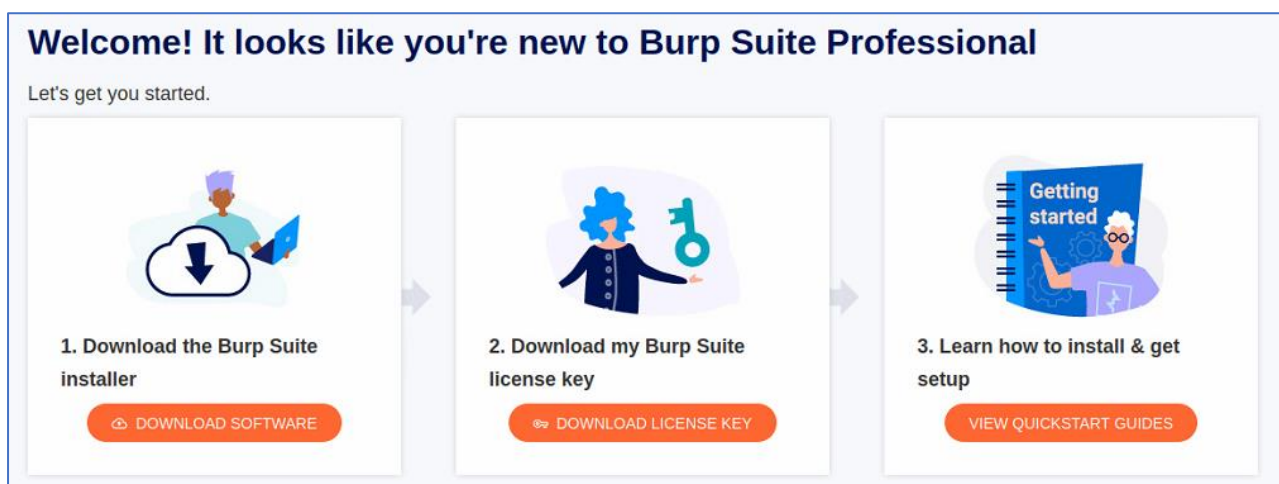


На этом установка OWASP Juice Shop завершена.

[Установка Burp Suite Professional](#)

Для выполнения практического задания мы будем использовать 30-дневную пробную версию, доступную для скачивания после регистрации на сайте разработчика <https://portswigger.net/burp/pro>

После регистрации, в своём кабинете скачиваем архив и лицензионный ключ:



Professional / Community 2023.9.4

Stable

01 September 2023 at 07:17 UTC

Burp Suite Professional

Linux (64-bit)

DOWNLOAD

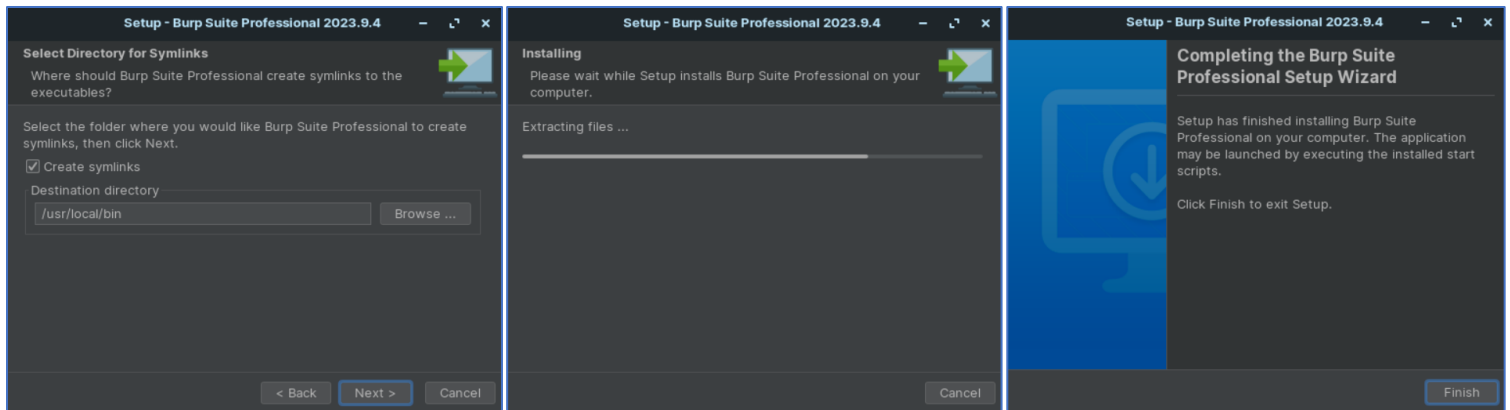
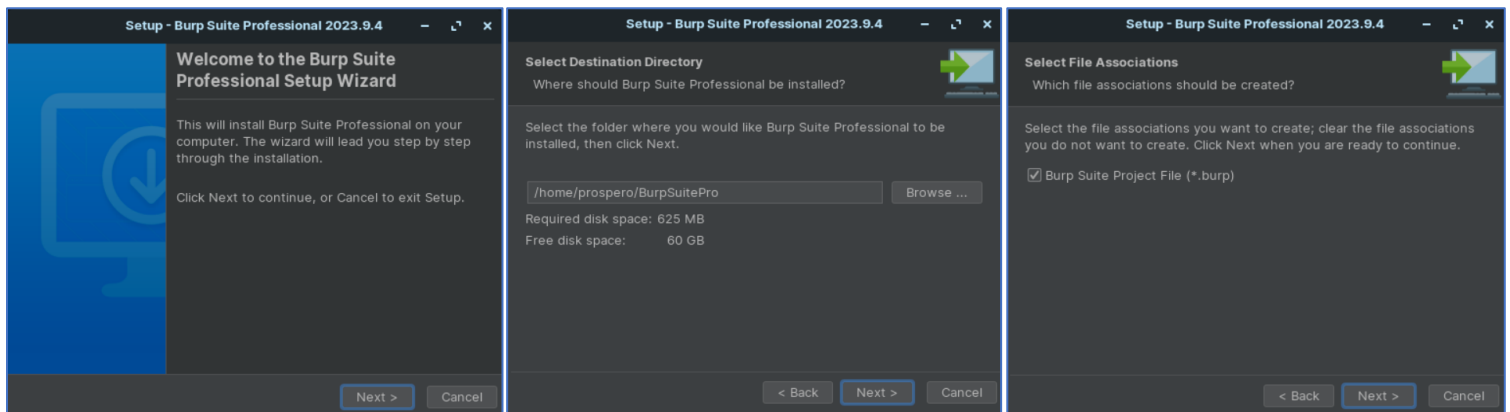
Установим права на запуск для скрипта установки командой:

```
$ chmod +x burpsuite_pro_linux_v2023_9_4.sh
```

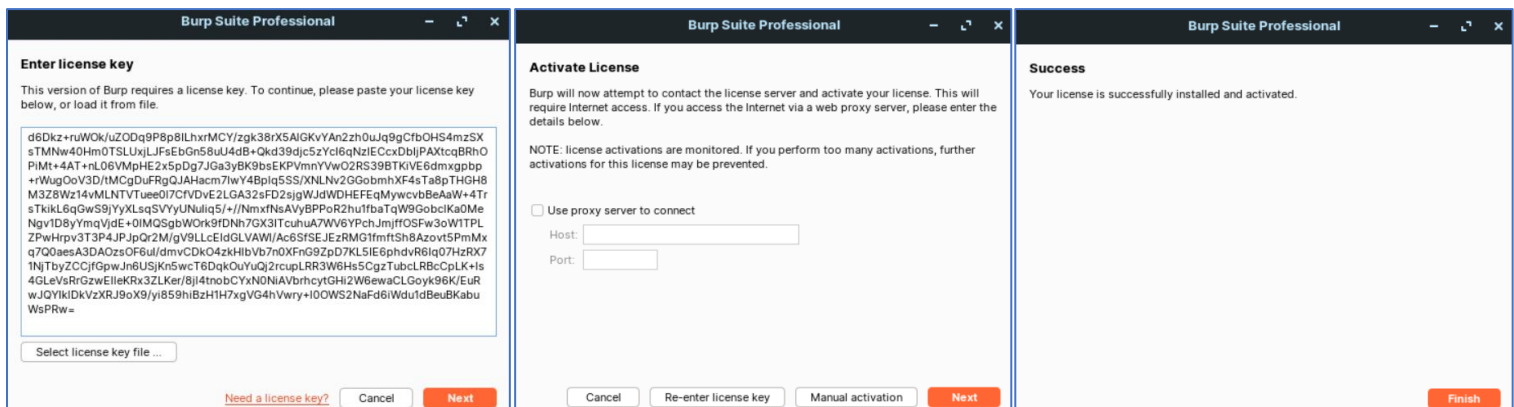
Далее запускаем скрипт установки и следуем указаниям мастера установки:

```
$ ./burpsuite_pro_linux_v2023_9_4.sh
```

```
prospero@zorin-vm:~/Downloads$ chmod +x burpsuite_pro_linux_v2023_9_4.sh
prospero@zorin-vm:~/Downloads$ ./burpsuite_pro_linux_v2023_9_4.sh
```



Запускаем программу, указываем путь к лицензионному ключу и активируем лицензию:



Создаём временный проект, нажимаем **Next**, используем дефолтные настройки конфигурации, жмем **Start Burp**:

Welcome to Burp Suite Professional. Use the options below to create or open a project.

Temporary project

New project on disk

Name:

File:

Choose file...

Open existing project

Name	File
------	------

File:

Choose file...

☐ Trust this project file

☐ Pause Automated Tasks

Cancel

Next

Select the configuration that you would like to load for this project.

Use Burp defaults

Use settings saved with project

Load from configuration file

File

File:

Choose file...

☐ Default to the above in future

☐ Disable extensions

Cancel

Back

Start Burp

Burp Suite Professional v2023.9.4 - Temporary Project - licensed to trial user

Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions **Learn** Settings

Learn, explore and discover

Hide this tab

Getting started with Burp Suite

Get going right away - with our quick start tutorial.

Start here



Burp Suite - a guided video tour

Take a run-through of all the major Burp Suite features.

Watch the tour



Burp Suite video tutorials

See how to use Burp Suite's main features and tools.

Find out more



The Web Security Academy

Learn how to find more vulnerabilities using Burp Suite.

Start learning



Burp Suite Support Center

Find the answers to your Burp Suite questions here.

Find answers



Burp Suite on Twitter

Join Burp Suite's huge community, and stay in the know.

Follow us



Мы успешно установили и активировали пробную версию Burp Suite Professional.

Установка FoxyProxy

Далее, установим плагин FoxyProxy для нашего браузера (Firefox):



Recommended

FoxyProxy Standard

by Eric H. Jung

FoxyProxy is an advanced proxy management tool that completely replaces Firefox's limited proxying capabilities. For a simpler tool and less advanced configuration options, please use FoxyProxy Basic.

Add to Firefox

Add FoxyProxy Standard? This extension will have permission to:

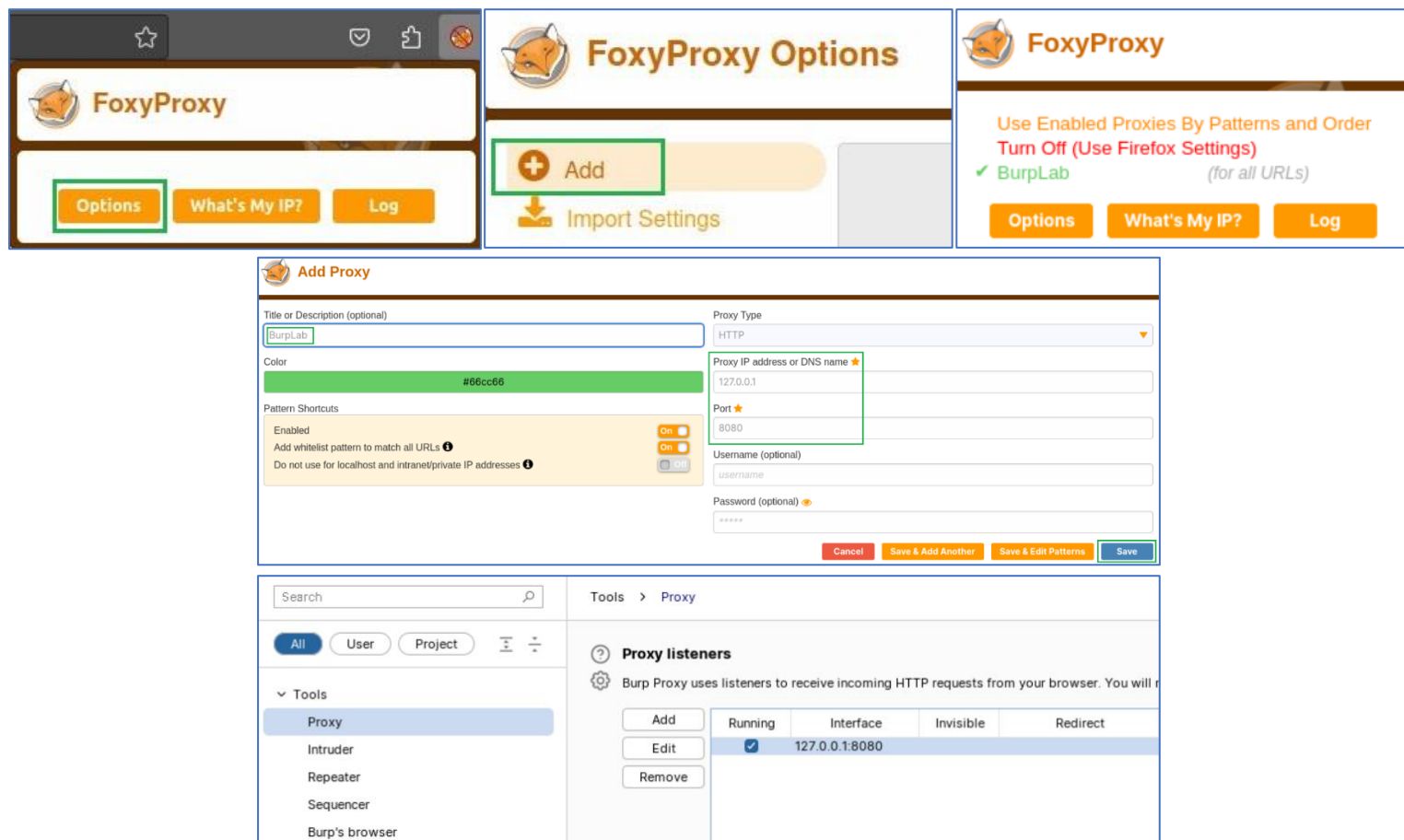
- Access your data for all websites
- Clear recent browsing history, cookies, and related data
- Download files and read and modify the browser's download history
- Display notifications to you
- Control browser proxy settings
- Access browser tabs

Learn more

Cancel

Add

Выполним настройку FoxyProxy:



Мы завершили установку и настройку всех необходимых нам инструментов!

ЭКСПЛУАТАЦИЯ УЯЗВИМОСТЕЙ ИЗ OWASP TOP-10:

1. **Authentication Bypass (Брешь в аутентификации):** представляет собой одну из наиболее критических уязвимостей, которые могут возникнуть в веб-приложениях. Она касается проблем, связанных с недостаточной или неправильной проверкой подлинности пользователей и механизмами аутентификации в приложениях. Эта уязвимость может привести к тому, что злоумышленник получит несанкционированный доступ к функциям, данным или действиям, на которые он не имеет права.

В рамках OWASP Top 10 категория "Брешь в аутентификации" включает в себя несколько подтипов или уязвимостей, связанных с аутентификацией и авторизацией:

- **Weak Authentication (Слабая аутентификация):** эта уязвимость возникает, когда механизм аутентификации слишком слаб или неадекватен для защиты ресурсов. Например, использование простых паролей без требований к их сложности или использование механизмов, которые не требуют подтверждения личности пользователя;
- **Session Attacks (Сессионные атаки):** злоумышленники могут перехватывать или подменять сессионные данные, что позволяет им войти в систему как другой пользователь без знания его пароля;
- **CAPTCHA Bypass:** может включать в себя обход системы CAPTCHA для выполнения действий, которые ограничены на основе успешной аутентификации или верификации пользователя как человека;
- **Недостаточные меры безопасности на стороне сервера:** в некоторых случаях, серверные компоненты могут допускать аутентификацию или выполнение функций без должной проверки подлинности.

Брешь в аутентификации — это серьезная уязвимость, так как она может привести к компрометации данных, утечке конфиденциальной информации и нарушению безопасности приложения. Эта категория уязвимостей выделяется в OWASP Top 10, чтобы подчеркнуть важность должной проверки подлинности пользователей и управления доступом.

Name	Difficulty	Description
CAPTCHA Bypass	★ ★ ★	Submit 10 or more customer feedbacks within 20 seconds.

Цель: обойти систему CAPTCHA и отправить 10 или более отзывов от клиентов за 20 секунд.

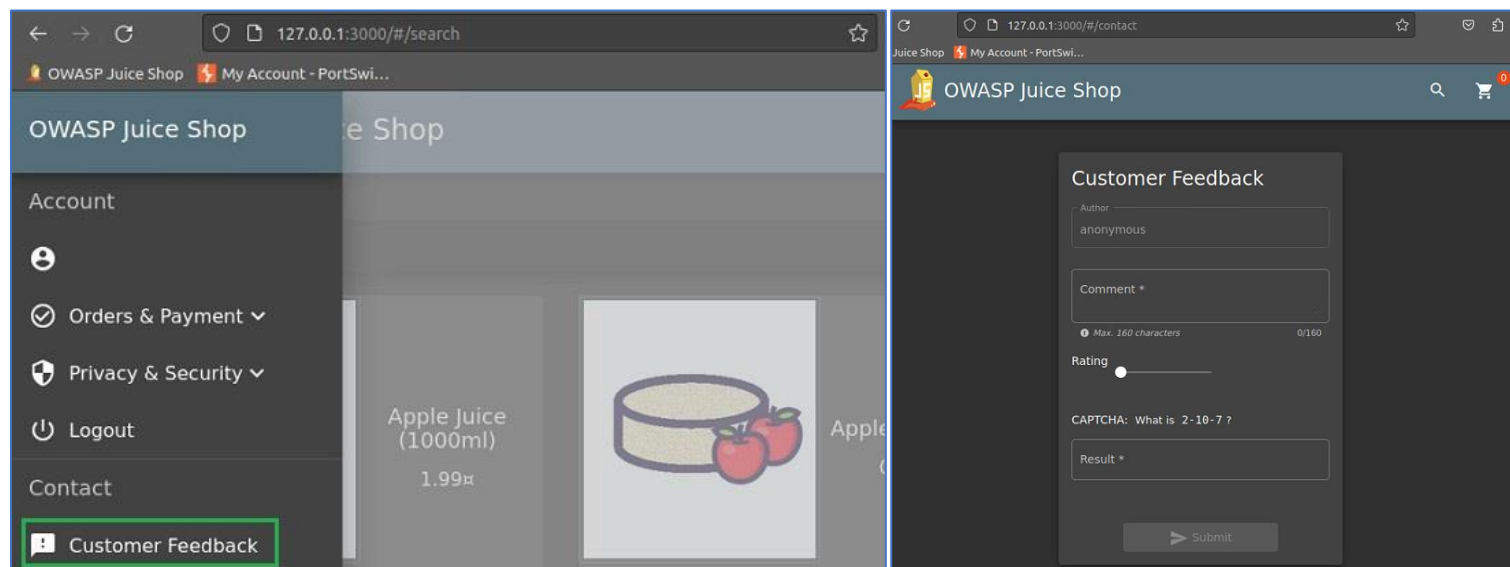
Задание "CAPTCHA Bypass" означает, что пользователь должен обойти механизм CAPTCHA, который обычно представляет собой проверочное изображение или пазл, который пользователь должен решить, чтобы доказать, что он человек. В этом контексте задача заключается в отправке 10 или более отзывов от клиентов на веб-сайте Juice Shop за очень короткое время - 20 секунд. Это позволяет продемонстрировать, что механизм CAPTCHA, предназначенный для защиты от автоматизированных атак, может быть обойден, что представляет потенциальную уязвимость в системе.

Это задание помогает обучить разработчиков и тестировщиков о важности адекватной защиты от ботов и скриптов, а также пониманию методов, которыми злоумышленники могут обходить меры безопасности.

Выполнение:

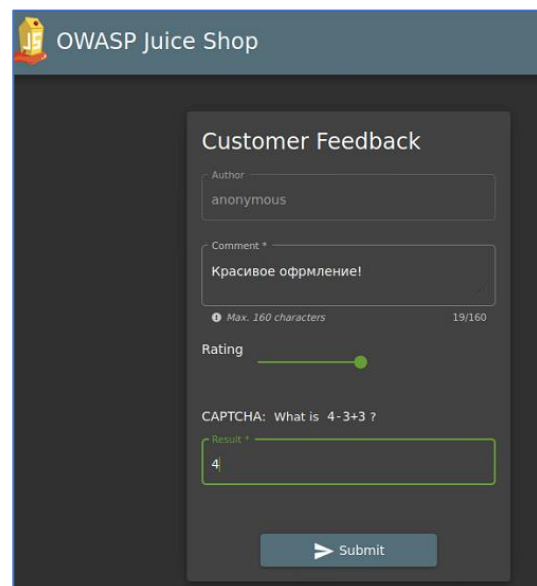
1. **Изучение сайта:** Первым шагом является изучение самого веб-сайта Juice Shop, чтобы понять, как работает механизм CAPTCHA и какие элементы он использует.

Переходим в раздел Customer Feedback:

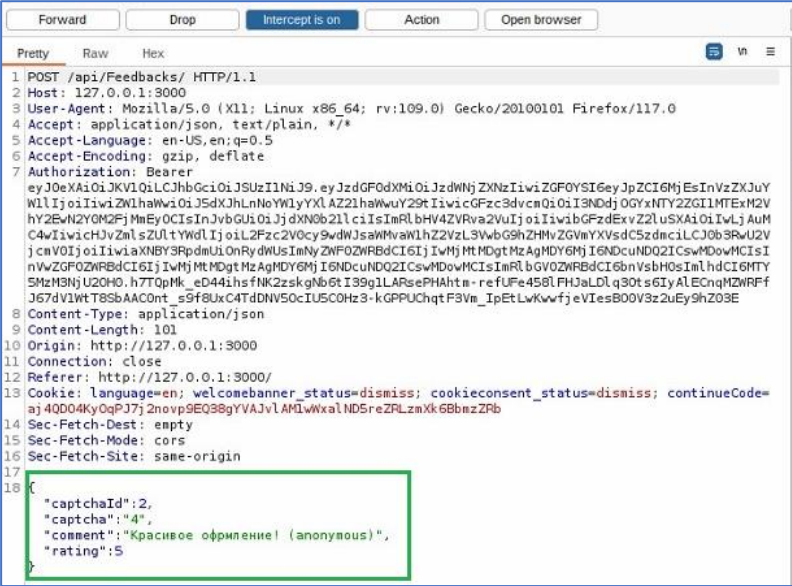


2. **Анализ CAPTCHA:** необходимо выяснить, каким образом реализован механизм CAPTCHA на сайте. Это может быть обычное изображение с текстом или другой вид проверки, такой как решение головоломки.

- В поле "Comment *" мы можем написать наш комментарий, например:
✓ Красивое оформление!
- Перетащить ползунок рейтинга (Rating), например:
✓ Рейтинг 5★
- Решить простое мат. вычисление и ввести ответ в поле "Result *":
✓ Результат: 4
- Нажать Отправить (Submit)

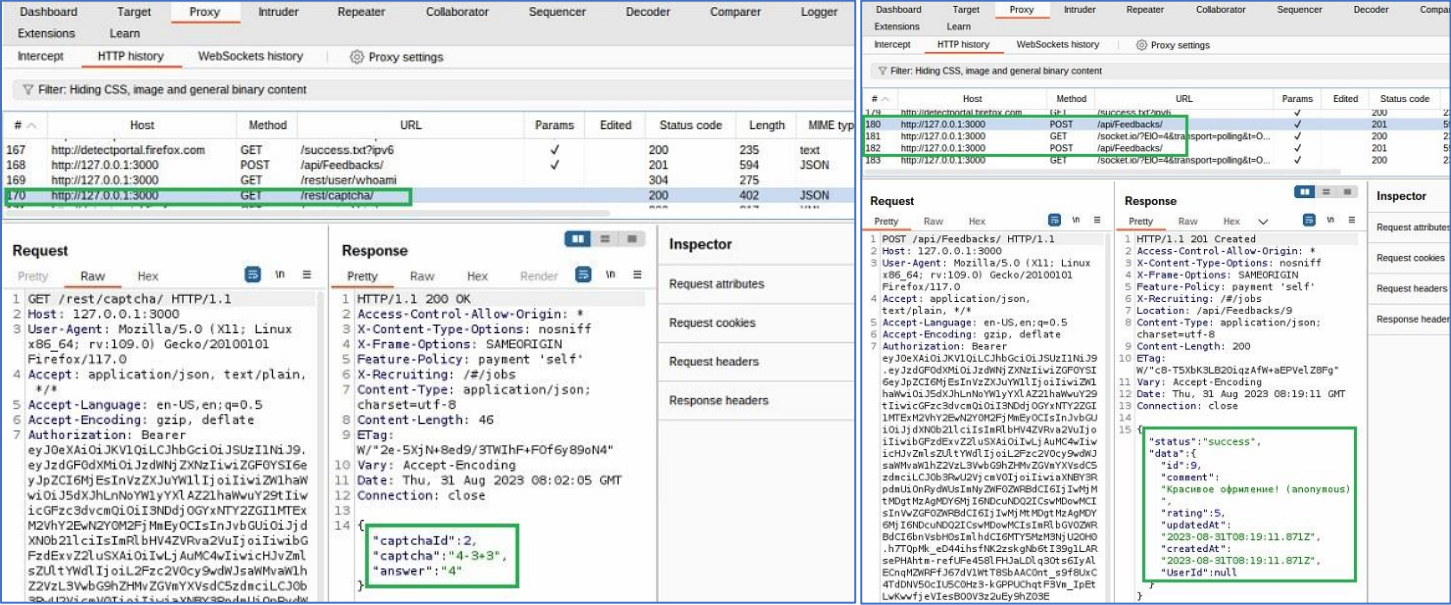


Мы перехватили наш отзыв в Intercept:

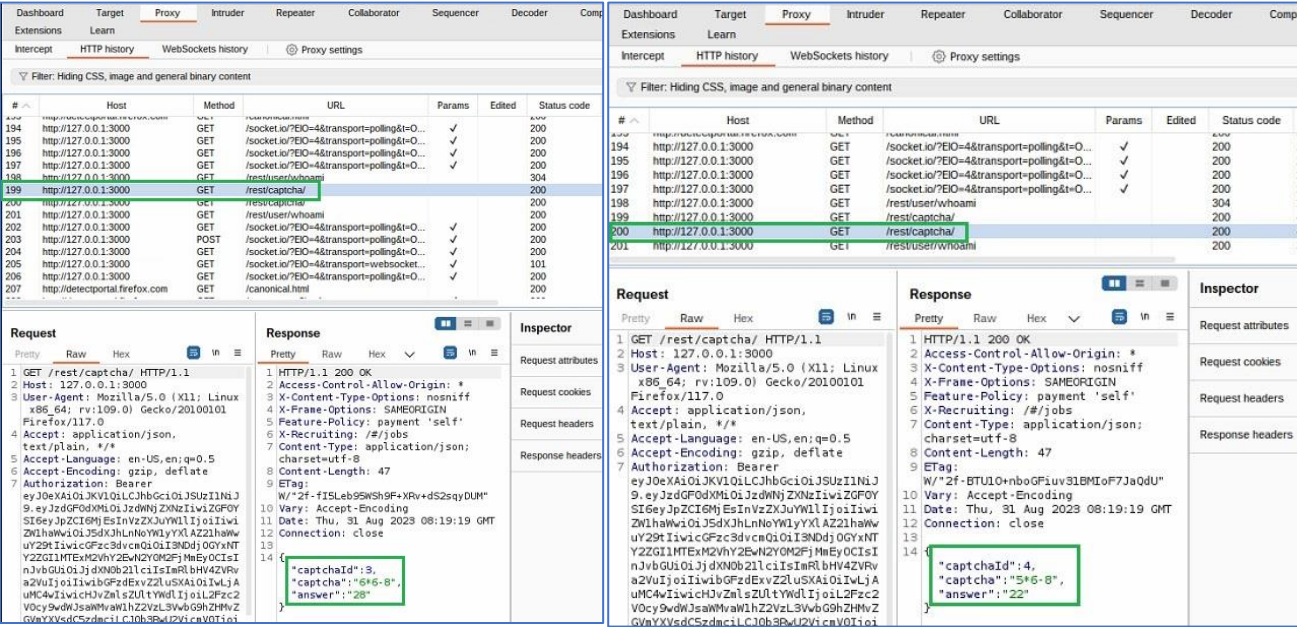


3. Поиск уязвимостей: анализируем код веб-страницы, ищем слабые места, которые могут быть использованы для обхода механизма CAPTCHA. Может быть, существует какой-то способ, позволяющий автоматизировано отправлять формы с отзывами без реального взаимодействия с CAPTCHA.

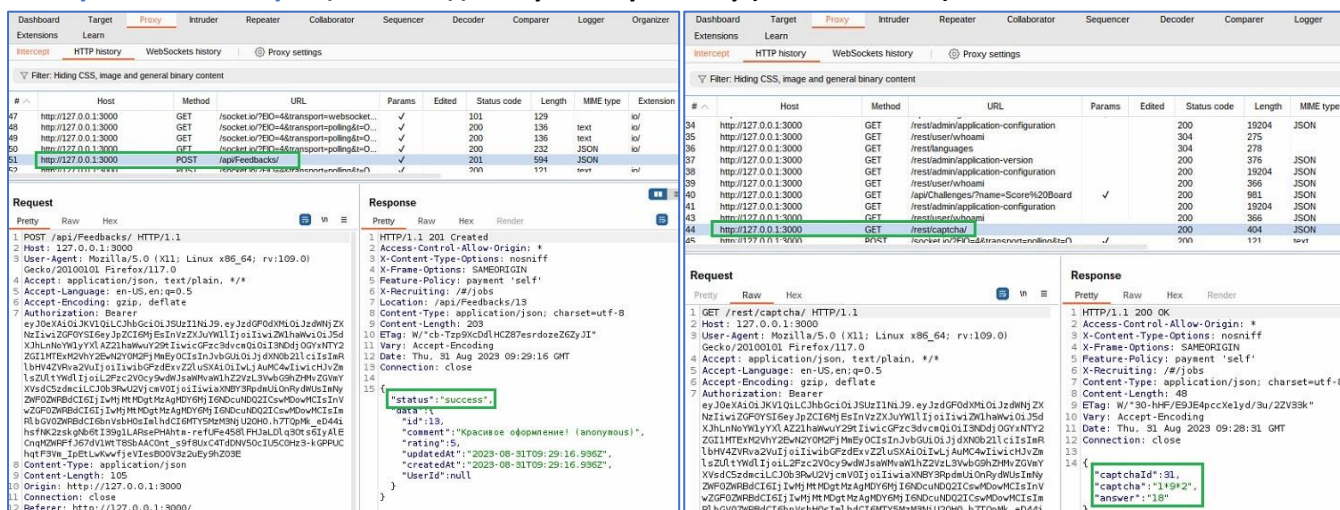
На вкладке HTTP History анализируем историю:



Видим, что после успешной отправки нашего отзыва (Капчи) нам в ответ «прилетает» сразу две новые капчи, также изменяется id для каждой капчи:



- ## Сгенерируем новый отзыв и перехватим нашу капчу:



В Repeater меняем параметры полей и многократно нажимаем кнопку **Send**:

- **captchald: 31**
- **captcha: 18**

The image shows a web application interface for 'OWASP Juice Shop'. At the top, a green banner reads: 'You successfully solved a challenge: CAPTCHA Bypass (Submit 10 or more customer feedbacks within 20 seconds.)'. Below this is a 'Customer Feedback' form with fields for 'Author' (set to 'anonymous'), 'Comment *', 'Rating' (set to 2), and 'CAPTCHA: What is 8-3*3?'. A 'Submit' button is at the bottom. On the left, a 'Request' log shows a POST request to '/api/Feedbacks/' with a JSON body:

```
{ "captchaId": 31, "captcha": "18", "comment": "Медленно загружается (anonymous)", "rating": 2 }
```

. The 'captchaId' and 'captcha' fields are highlighted with a green box.

6. **Подтверждение уязвимости:** после успешного выполнения задания у нас есть доказательство того, что механизм CAPTCHA может быть обойден.

A dark banner with the text 'CAPTCHA Bypass' on the left, three stars in the center, and 'Submit 10 or more customer feedbacks within 20 seconds.' on the right. A green checkmark icon is in a box on the far right.

Рекомендации по устранению:

- Улучшение CAPTCHA реализации, т. е. использование более сложных и разнообразных тип CAPTCHA, которые сложнее обойти автоматически;
- Внедрение дополнительных проверок и анализа, чтобы определить, является ли пользователь истинным человеком или ботом, помимо CAPTCHA. Это может включать в себя анализ поведения пользователя, реакцию на ввод и другие метрики;
- Ведение журнала попыток ввода CAPTCHA и их результатов для детального анализа и обнаружения подозрительной активности;
- Разработка и внедрение системы блокировки или ограничения доступа для IP-адресов, совершивших множественные неудачные попытки обхода CAPTCHA;
- Проведение регулярных аудитов безопасности приложения, включая тестирование на уязвимости CAPTCHA Bypass;
- Используйте современные инструменты для анализа безопасности, чтобы выявить слабые места в защите от CAPTCHA Bypass.

Устранение уязвимости CAPTCHA Bypass требует комплексного подхода, который включает в себя технические улучшения, мониторинг и постоянное обновление мер безопасности. Регулярные аудиты и тестирование безопасности также важны для обнаружения и предотвращения новых методов атак.

2. **Injection (Ињекции):** в рамках OWASP Top 10 эта категория описывает уязвимости, связанные с внедрением вредоносных данных или кода в систему с целью выполнения неконтролируемых операций на стороне сервера, включая базы данных.

Ињекции относятся к классу уязвимостей, при которых злоумышленник вводит вредоносные данные, которые затем "внедряются" или интерпретируются как код на стороне приложения или сервера. Это может включать в себя SQL ињекции, командные ињекции (Command Injection), ињекции кода JavaScript (Cross-Site Scripting, XSS), и другие формы атак.

Ињекции являются критической угрозой для безопасности веб-приложений, потому что они могут привести к:

- **Утечке данных:** злоумышленник может получить доступ к чувствительным данным, таким как личная информация, пароли, кредитные карты и другие конфиденциальные сведения;
- **Нарушению безопасности:** ињекции могут использоваться для обхода механизмов аутентификации и авторизации, что позволяет злоумышленникам выполнять действия с правами доступа, которые им не полагаются.
- **Отказе в обслуживании (DoS):** неконтролируемые операции могут привести к перегрузке сервера и отказу в обслуживании.

Name	Difficulty	Description
Database Schema	★ ★ ★	Exfiltrate the entire DB schema definition via SQL Injection.

Цель: Используя метод SQL ињекции, извлечь и скопировать полное определение схемы базы данных.

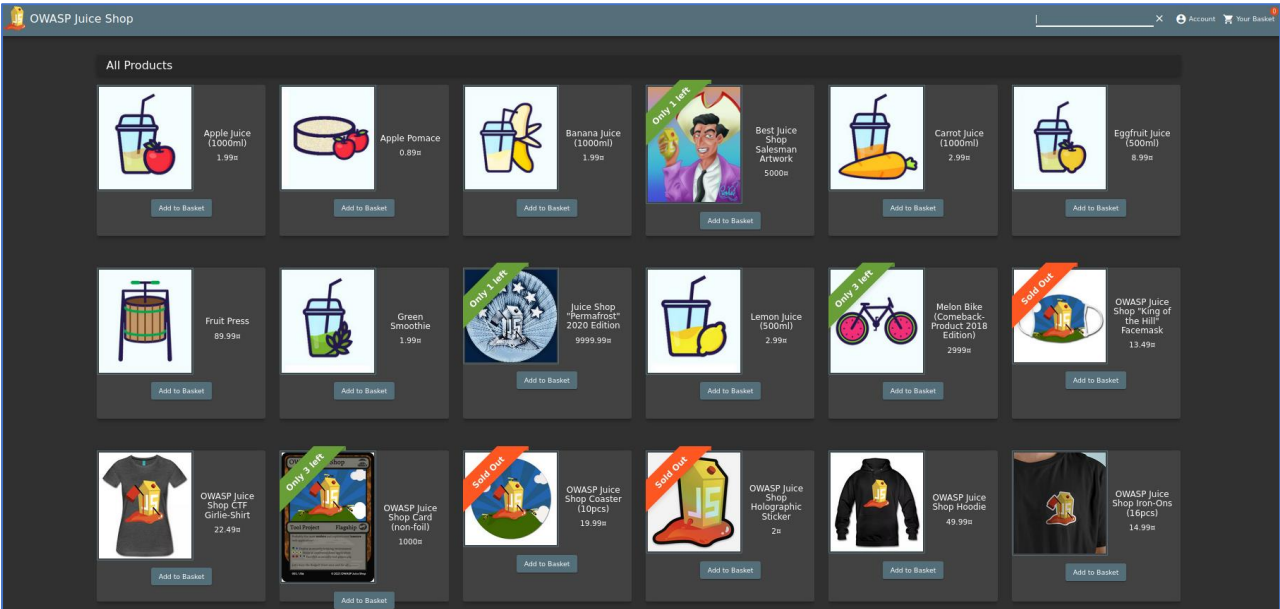
В этой задаче демонстрируется уязвимость веб-приложения, связанная с SQL ињекциями, т. е. злоумышленник может использовать ввод, чтобы извлечь определение схемы всей базы данных. Схема базы данных — это структура, определяющая таблицы, столбцы, типы данных и отношения между ними. Злоумышленник может использовать такие атаки, чтобы извлечь ценную информацию о структуре базы данных, которая может быть использована для последующих атак или эксплуатации.

Эта задача предоставляет возможность тестировщикам, разработчикам и другим специалистам по информационной безопасности понять, как работают SQL ињекции, и как важно обеспечивать безопасность ввода данных, чтобы предотвратить подобные атаки.

Выполнение:

1. **Исследование приложения:** снова начинаем с изучения веб-сайта Juice Shop, пройдём по различным разделам, чтобы понять какие формы, вводы и функции доступны на сайте.

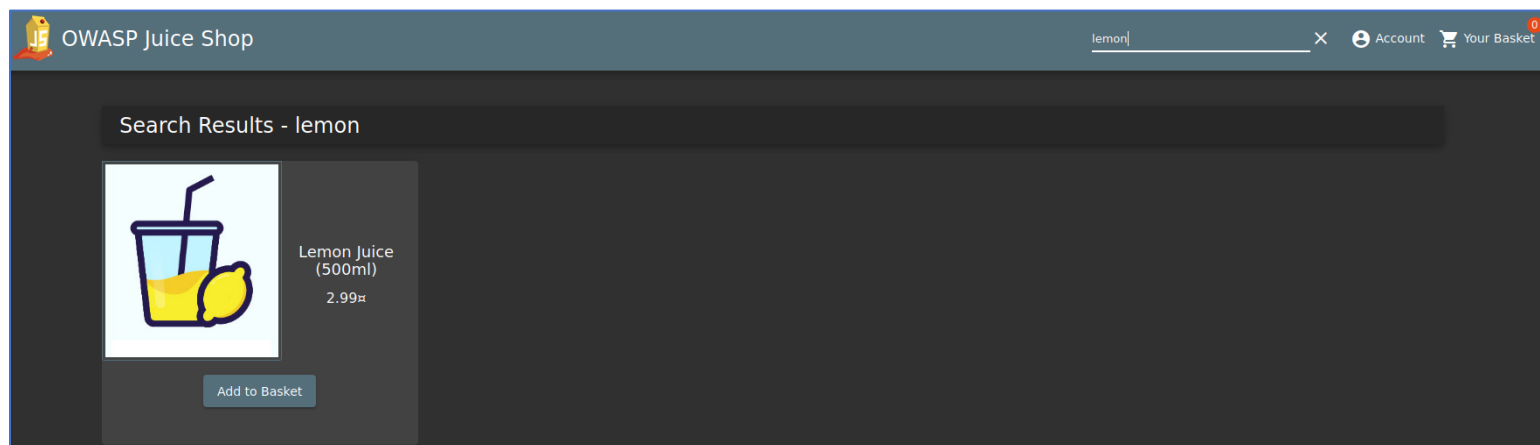
Перейдем на стартовую страницу:



Мы видим каталог продуктов, где есть их: название, цена, наличие/отсутствие продукта и многое другое. Все эти данные хранятся в базе данных веб-сайта.

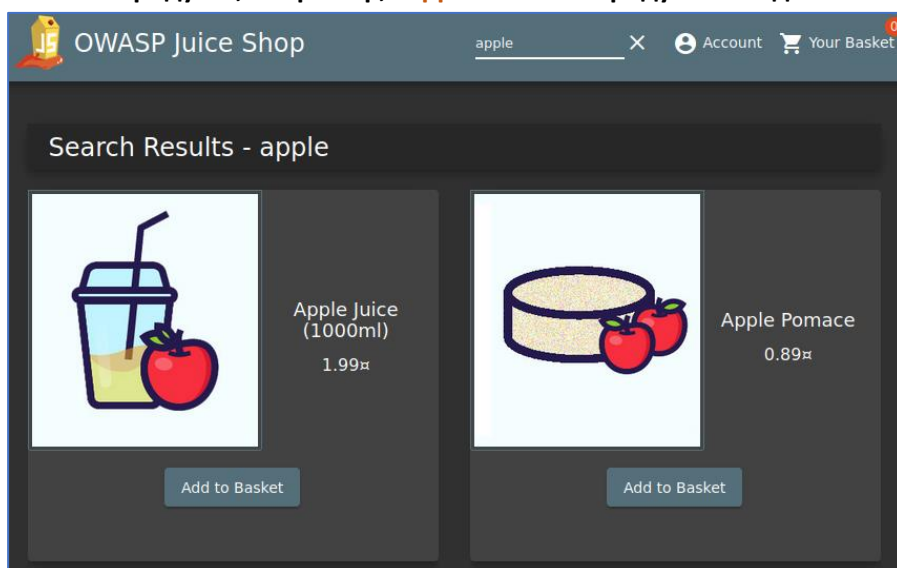
2. Поиск уязвимого места: ищем форму или место, где пользовательский ввод может быть использован в SQL-запросе. Попробуем понять, какой вид SQL-запроса используется для взаимодействия с базой данных.

В верхней правой части веб-станции мы видим поле ввода для поиска по сайту, где мы можем ввести название какого-нибудь продукта. Попробуем ввести название продукта, например мы ищем Лимонный Сок, вводим **“lemon”**

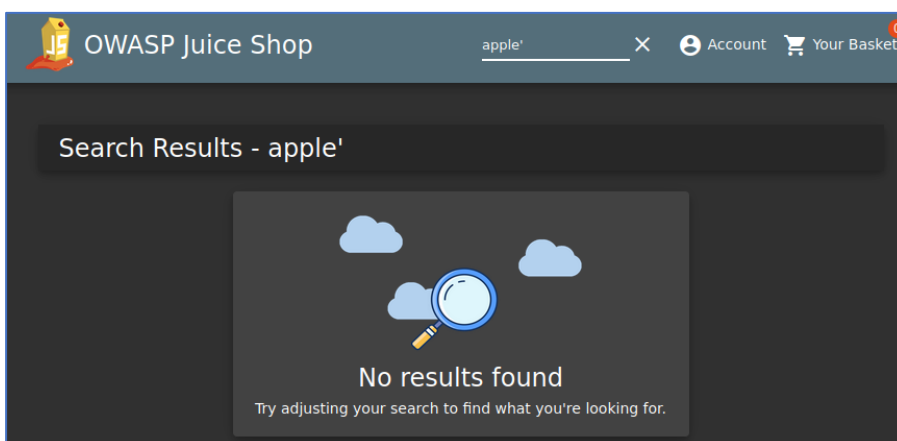


Продукт с таким названием есть в базе магазина.

Теперь вводим другое название продукта, например, **“apple”** – такие продукты найдены в базе:



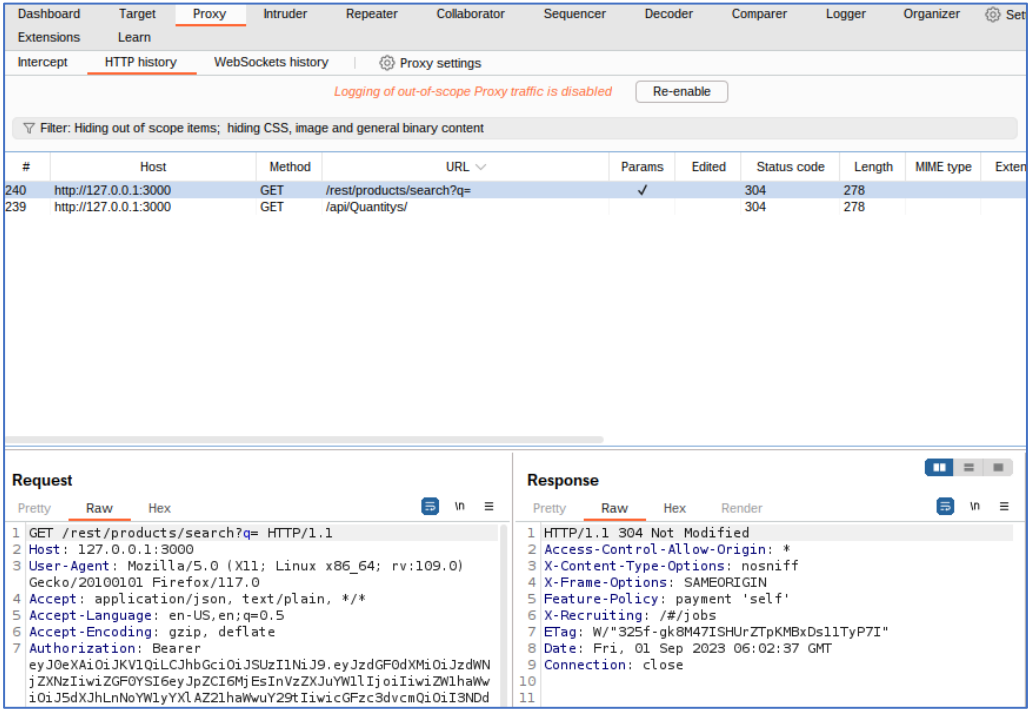
Теперь проверим, что будет если в строке поиска добавить к названию какой-нибудь символ, например одинарные кавычки, **apple'**:



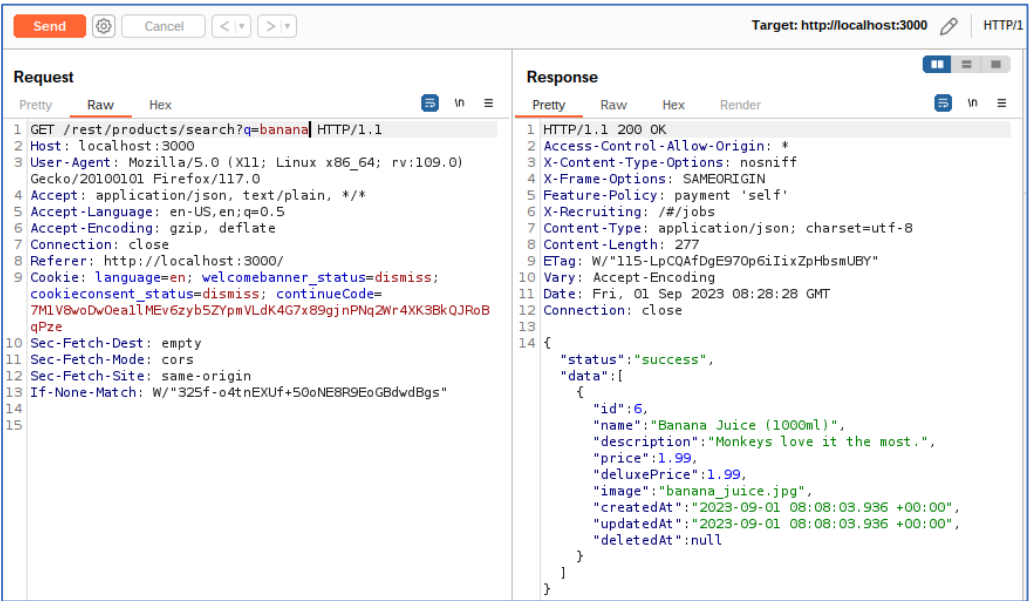
Получили сообщение (но не ошибку), что ничего не найдено.

Теперь нам надо внимательно изучить весь процесс отправки запросов и получения ответов в Burp Suite.

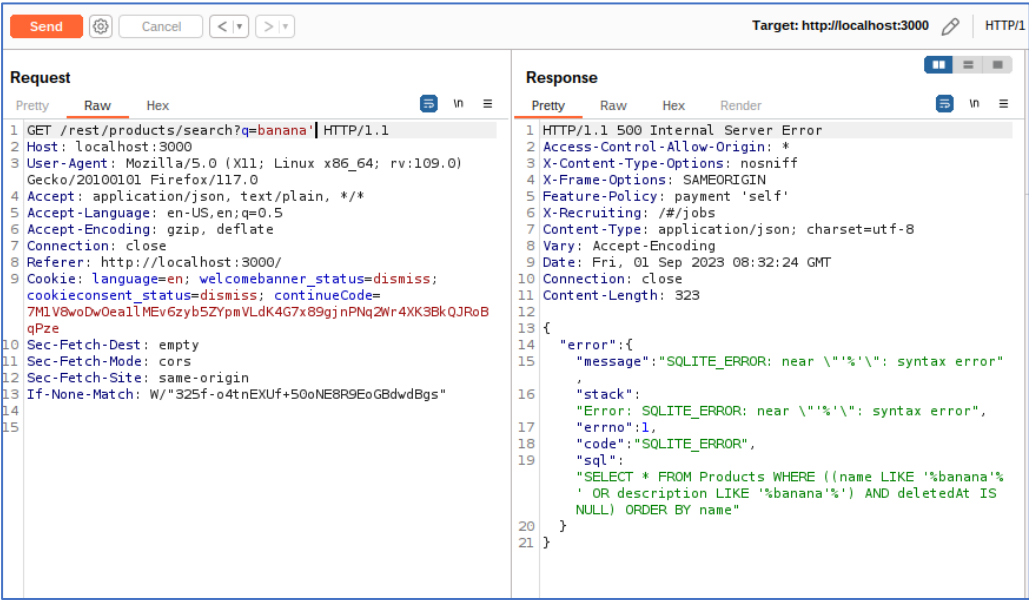
Мы видим запрос с параметрами “q= ” т. е. отсутствующие данные, которые мы вводили в строке поиска:



Теперь отправим наш запрос в Repeater и изменим данные, например “q=banana”:



Теперь отправим такой же запрос, но поставим после названия символ – banana’



Мы получили ошибку “SQLITE_ERROR, которая сообщает нам не только об ошибке в запросе, но и какая база данных используется (SQLite).

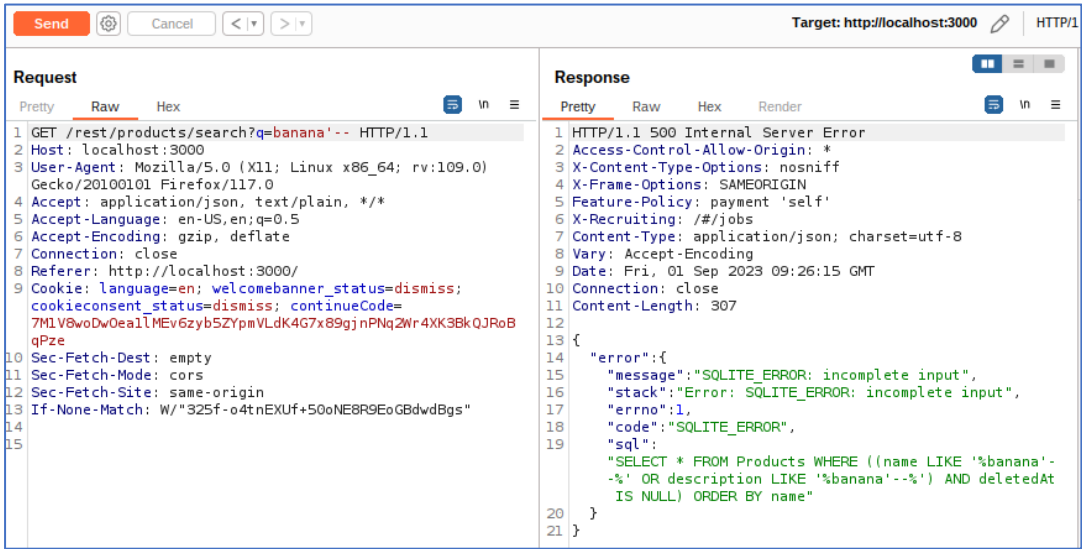
3. Понимание схемы базы данных: необходимо ознакомиться со структурой базы данных Juice Shop, если это возможно. Нам поможет документация Juice Shop или анализ запросов к базе данных.

Далее, нам понадобится установить SQLite3, выполним команду в терминале:
\$ sudo apt install sqlite3

SQLite3 — это версия для командной строки, которая позволяет пользователям управлять базами данных SQLite и выполнять SQL-запросы непосредственно из командной строки. Подробное описание работы с SQLite3 выходит за рамки нашего практического задания, поэтому всю дополнительную информацию можно изучить на официальном сайте или с помощью поисковика Google.

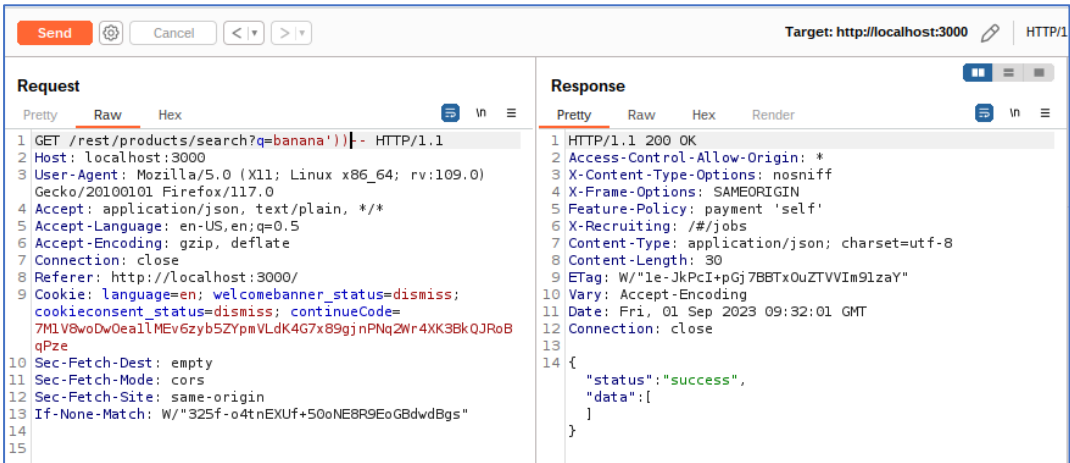
4. Проведение SQL-инъекции: используем техники SQL-инъекции, чтобы внедрить злоумышленный код в запрос. Попробуем изменить запрос таким образом, чтобы он возвращал определение схемы базы данных, используя команды SQL.

Возвращаемся в наш Repeater, используем различные символы, например q=banana’--



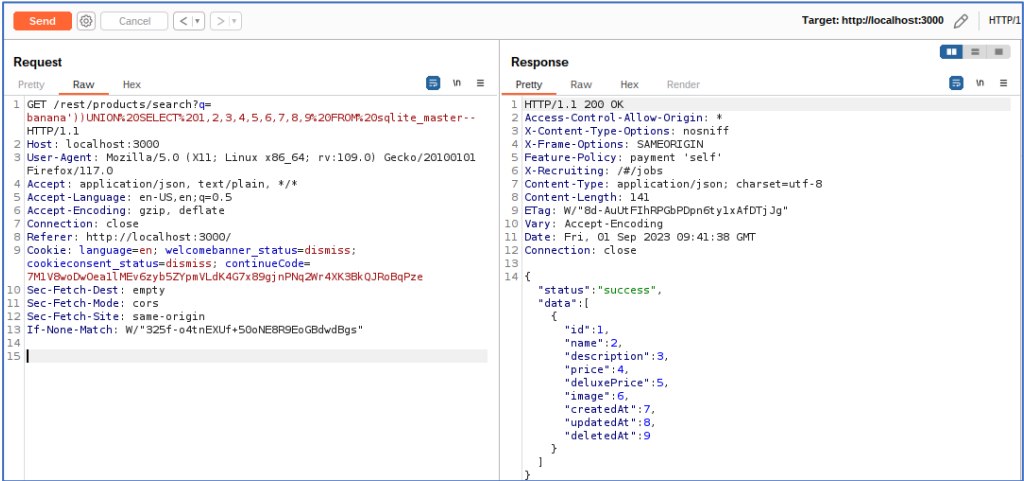
Получили ошибку, что ввод не завершен - “SQLITE_ERROR: incomplete input”

Также мы видим, что возможно нам надо ввести в нашем запросе две закрывающие скобки `)`, пробуем добавить: q=banana’)))- -



Получили status: success

Теперь пробуем вставить SQL код с использованием комментария SQL – **UNION** (SELECT или других конструкций): **banana'))UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--**



Подробно рассмотрим, что делает каждая часть этой SQL-команды:

UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master: это вставка SQL-кода, который пытается выполнить операцию объединения таблиц. Оператор **UNION** используется для объединения результатов двух SQL-запросов в один результат. В данном случае попытка объединения с таблицей **sqlite_master**, которая содержит информацию о структуре базы данных.

-- это комментарий в SQL, который означает, что всё, что находится после **--**, будет игнорироваться сервером базы данных. Это часто используется злоумышленниками для закрытия строки запроса и предотвращения выполнения дополнительного текста запроса.

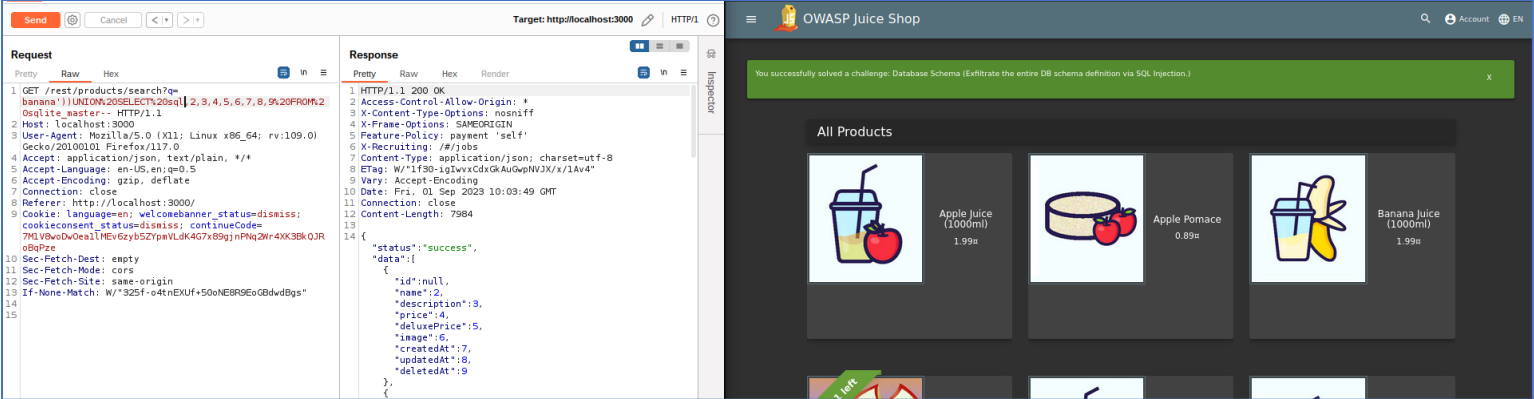
Проанализируем ответ, который мы получили:

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": 2,
      "description": 3,
      "price": 4,
      "deluxePrice": 5,
      "image": 6,
      "createdAt": 7,
      "updatedAt": 8,
      "deletedAt": 9
    }
  ]
}
```

Исходя из предоставленных данных, это может быть воспринято как таблица или, точнее, как результат SQL-запроса, который содержит структуру и данные из таблицы базы данных.

Теперь пробуем вставить SQL-код в поле ввода таким образом, чтобы мы могли извлечь определение схемы базы данных:

banana'))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--



Также, в ответе видим всю схему базы данных:

Response

ProxyRawHexRender

```
<
{
  "id": "CREATE TABLE 'Addresses' ('userId' INTEGER REFERENCES 'Users' ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'fullName' VARCHAR(255), 'mobileNum' INTEGER, 'zipCode' VARCHAR(255), 'streetAddress' VARCHAR(255), 'city' VARCHAR(255), 'state' VARCHAR(255), 'country' VARCHAR(255), 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL);",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'BasketItems' ('productId' INTEGER REFERENCES 'Products' ('id') ON DELETE CASCADE ON UPDATE CASCADE, 'basketId' INTEGER REFERENCES 'Baskets' ('id') ON DELETE CASCADE ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'quantity' INTEGER, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL, UNIQUE ('productId', 'basketId'))",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Baskets' ('id' INTEGER PRIMARY KEY AUTOINCREMENT, 'coupon' VARCHAR(255), 'userId' INTEGER REFERENCES 'Users' ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Captchas' ('id' INTEGER PRIMARY KEY AUTOINCREMENT, 'captchaId' INTEGER, 'captcha' VARCHAR(255), 'answer' VARCHAR(255), 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Users' ('id' INTEGER PRIMARY KEY AUTOINCREMENT, 'username' VARCHAR(255) DEFAULT '', 'email' VARCHAR(255) UNIQUE, 'password' VARCHAR(255), 'role' VARCHAR(255) DEFAULT 'customer', 'deluxeToken' VARCHAR(255) DEFAULT '', 'lastLoginIp' VARCHAR(255) DEFAULT '0.0.0.0', 'profileImage' VARCHAR(255) DEFAULT '/assets/public/loop-on/loopOnDefault.png', 'totpSecret' VARCHAR(255) DEFAULT '', 'isActive' TINYINT(1) DEFAULT 1, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL, 'deletedAt' DATETIME)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Wallets' ('userId' INTEGER REFERENCES 'Users' ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'balance' INTEGER DEFAULT 0, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE sqlite_sequence(name,seq)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
}
```

Response

ProxyRawHexRender

```
{
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Users' ('id' INTEGER PRIMARY KEY AUTOINCREMENT, 'username' VARCHAR(255) DEFAULT '', 'email' VARCHAR(255) UNIQUE, 'password' VARCHAR(255), 'role' VARCHAR(255) DEFAULT 'customer', 'deluxeToken' VARCHAR(255) DEFAULT '', 'lastLoginIp' VARCHAR(255) DEFAULT '0.0.0.0', 'profileImage' VARCHAR(255) DEFAULT '/assets/public/loop-on/loopOnDefault.png', 'totpSecret' VARCHAR(255) DEFAULT '', 'isActive' TINYINT(1) DEFAULT 1, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL, 'deletedAt' DATETIME)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE 'Wallets' ('userId' INTEGER REFERENCES 'Users' ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'balance' INTEGER DEFAULT 0, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
{
  "id": "CREATE TABLE sqlite_sequence(name,seq)",
  "name": "2",
  "description": "3",
  "price": "4",
  "deluxePrice": "5",
  "image": "6",
  "createdAt": "7",
  "updatedAt": "8",
  "deletedAt": "9"
},
}
```

Database Schema★ ★ ★Exfiltrate the entire DB schema definition via SQL Injection.Injection

✓ solved

Рекомендации по устранению:

- Адекватно фильтровать и экранировать входные данные, чтобы не допустить внедрения вредоносного кода;
- Использовать параметризованные запросы в базе данных, вместо конкатенации строк, чтобы избежать SQL инъекций;
- Регулярно обновлять и патчить программное обеспечение и библиотеки, чтобы устранить известные уязвимости.

Уязвимости в категории "Инъекции" остаются одной из наиболее актуальных и опасных угроз в области безопасности веб-приложений, и они широко учитываются в рамках OWASP Top 10 для повышения осведомленности и безопасности разработки приложений.

3. Authentication Bypass & Injection (Брешь в аутентификации + Инъекции):

Name	Difficulty	Description
Login Jim	★ ★ ★	Log in with Jim's user account.

Цель задания "Login Jim" заключается в демонстрации и понимании уязвимости веб-приложения, связанной с инъекциями, а именно инъекциями в механизм аутентификации. Задача направлена на взлом аккаунта пользователя по имени "Jim" с использованием уязвимости инъекций в процессе аутентификации.

Как правило, уязвимости веб-приложений, связанные с инъекциями, возникают, когда пользовательский ввод недостаточно фильтруется или экранируется перед использованием в SQL-запросах, командах операционной системы или других контекстах. Злоумышленники могут использовать эту уязвимость для выполнения злонамеренных действий, таких как взлом аккаунтов, обход аутентификации или даже выполнения произвольных команд.

В данном случае цель задания "Login Jim" - воспроизвести атаку, используя уязвимость инъекций, чтобы успешно авторизоваться в системе от имени пользователя "Jim" без настоящего пароля или другой формы аутентификации. Это помогает разработчикам и тестировщикам безопасности лучше понять, какие меры безопасности не были приняты в приложении, и каким образом можно укрепить защиту от подобных инъекций и атак.

Выполнение:

1. Понимание уязвимости: В этой задаче предполагается, что существует уязвимость инъекции в механизме аутентификации. Нам нужно выяснить, где и как она может быть использована.

В предыдущем пункте мы извлекли схему базы данных и обнаружили таблицу “Users”, теперь попробуем с помощью SQL-запроса извлечь данные из этой таблицы и получить список всех пользователей.



2. Подготовка ввода: попробуем изменить данные в Repeater в нашем запросе:

q=qwert'))%20UNION%20SELECT%20id,email,password,'4','5','6','7','8','9'%20FROM%20Users--

Рассмотрим, что мы изменили в нашем запросе:

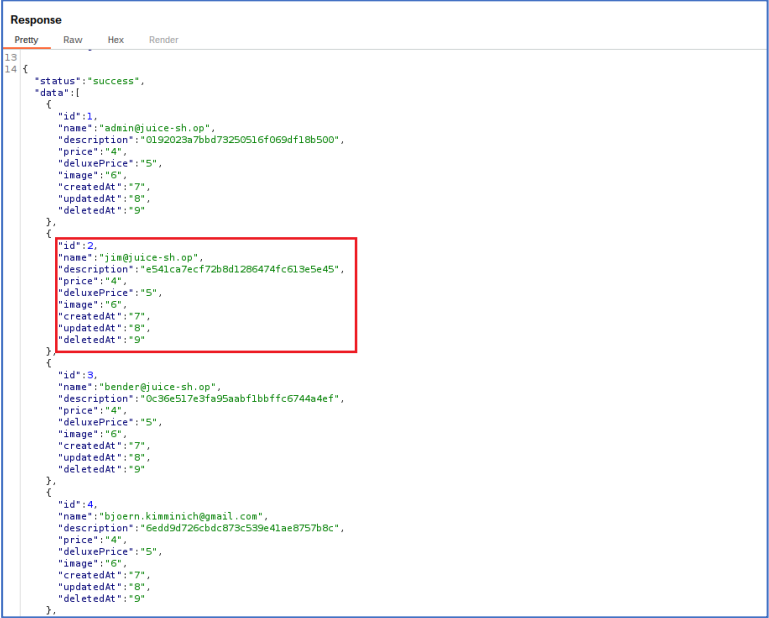
q=qwert': произвольная часть запроса, которая предназначена для поиска.

id,email,password: это столбцы, которые выбираются из таблицы Users в базе данных. Мы пытаемся извлечь информацию о пользователях, включая их идентификаторы (id), адреса электронной почты (email) и пароли (password).

Затем следуют строки с числами и символами, такими как '4', '5', и так далее. Они используются для заполнения оставшихся столбцов, чтобы соответствовать структуре результата предыдущего запроса.

FROM%20Users указывает, что выборка производится из таблицы Users.

После отправки запроса получаем ответ:



Мы видим, что в списке пользователей есть наша цель, пользователь с id:2, name: jim@juice-sh.op, (зарегистрирован с адресом электронной почты) также, в поле description данные "e541ca7ecf72b8d1286474fc613e5e45" похожи на хэш (пароль) его учетной записи.

Далее, воспользуемся онлайн-сервисом CrackStation.

CrackStation — это онлайн-инструмент и сервис для поиска и расшифровки хэшей паролей. Он предоставляет базу данных, содержащую миллионы заранее вычисленных хэшей паролей и их соответствующих исходных значений.

Вот основные характеристики CrackStation:

- **Поиск хэшей паролей:** CrackStation позволяет пользователям вводить хэши паролей, которые им нужно взломать, и затем ищет соответствующие им исходные пароли в своей базе данных. Если соответствие найдено, он возвращает исходный пароль;
- **Большая база данных:** CrackStation содержит огромную базу данных с миллионами хэшей паролей, предварительно вычисленными и упорядоченными для более быстрого поиска;
- **Поддержка разных хэш-алгоритмов:** сервис поддерживает различные алгоритмы хэширования, такие как MD5, SHA-1, SHA-256 и другие. Это позволяет искать пароли, зашифрованные разными алгоритмами;
- **Анонимность:** CrackStation не требует регистрации или входа, и пользователи могут анонимно вводить хэши паролей для поиска;
- **Образцы паролей:** кроме того, CrackStation предоставляет образцы паролей, которые могут быть использованы для улучшения безопасности паролей, и проверки на стойкость;
- **Обновления базы данных:** База данных CrackStation периодически обновляется, чтобы включить новые пароли и хэши, а также удалить старые;
- **Этичное использование:** помимо взлома паролей, CrackStation может быть использован в образовательных целях и для тестирования безопасности с разрешения владельца системы.

Переходим по ссылке <https://crackstation.net/>, вводим наш целевой хэш и нажимаем Crack Hashes:



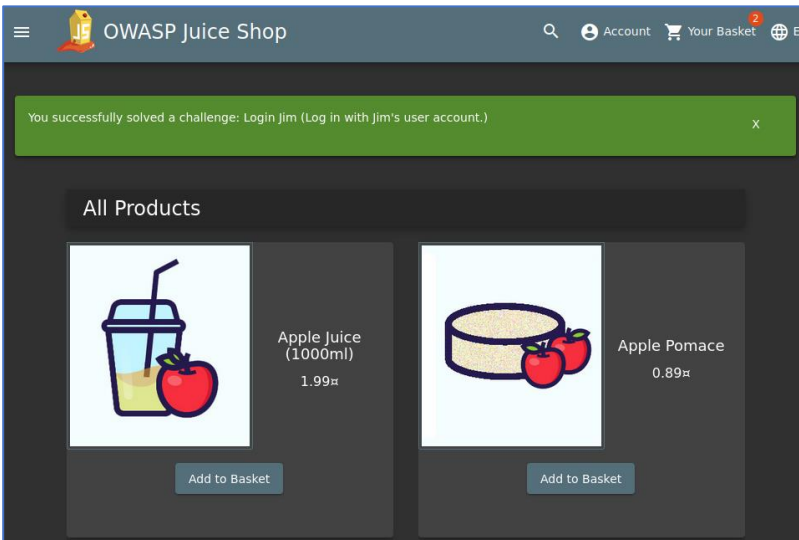
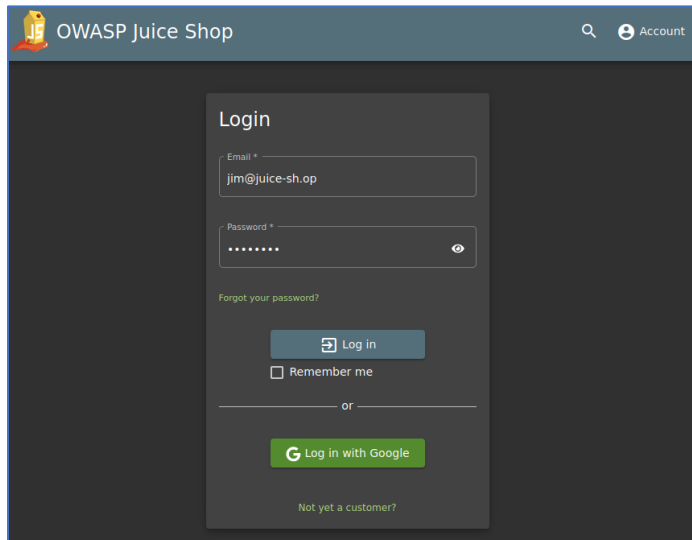
Мы получили результат – **ncc-1701** (это и является паролем для Jim)

3. Попробуем ввести имя пользователя и пароль:

вернемся на страницу входа и введем учетные данные пользователя Jim.

Email: **jim@juice-sh.op**

Password: **ncc-1701**





Рекомендации по устранению:

- Проведение адекватной фильтрации и экранирования входных данных, перед тем как использовать их в механизме аутентификации, чтобы предотвратить инъекции;
- Разработка аутентификации с использованием проверенных и безопасных библиотек и фреймворков, которые предоставляют встроенные механизмы аутентификации и авторизации;
- Реализация многофакторной аутентификации (MFA), чтобы усложнить процесс взлома аккаунтов даже при успешной аутентификации;
- Создание системы мониторинга, которая будет отслеживать необычную активность в процессе аутентификации и оповещать о потенциальных атаках;
- Проведение регулярного тестирования на уязвимости, включая инъекции, в механизме аутентификации, чтобы выявлять и устранять слабые места;
- Регулярное обновление компонентов и библиотек, используемых в механизме аутентификации, чтобы устранять известные уязвимости.

Устранение уязвимости в механизме аутентификации и профилактика инъекций важны для обеспечения безопасности веб-приложения и защиты аккаунтов пользователей от несанкционированного доступа. Это требует системного и многоуровневого подхода к обеспечению безопасности и регулярного обновления мер безопасности.

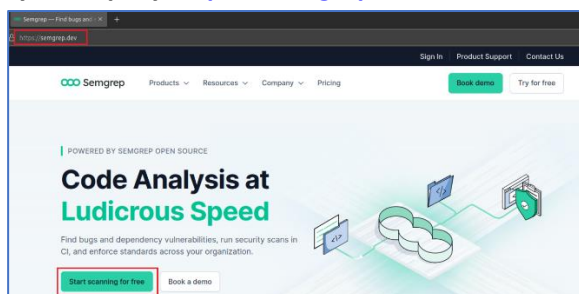
СТАТИЧЕСКИЙ АНАЛИЗ OWASP JUICE SHOP

Далее, в рамках нашего практического задания мы проведем статический анализ OWASP Juice Shop с помощью Semgrep на наличие уязвимостей. Это позволит нам выявить широкий спектр потенциальных угроз безопасности.

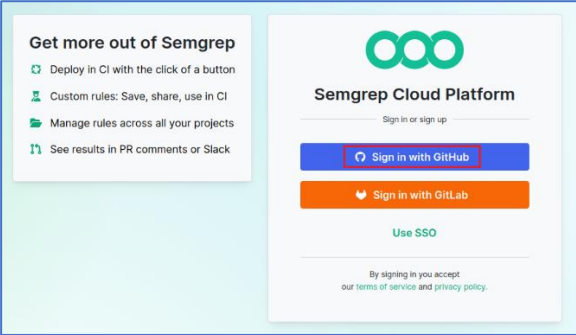
Semgrep — это мощный инструмент для статического анализа кода, который позволяет обнаруживать потенциальные уязвимости и нарушения правил безопасности в исходном коде приложений. Этот инструмент особенно полезен для обеспечения безопасности и качества кода на ранних этапах разработки, что позволяет предотвращать уязвимости и ошибки до их выхода в продакшен.

УСТАНОВКА и НАСТРОЙКА SEMGREP

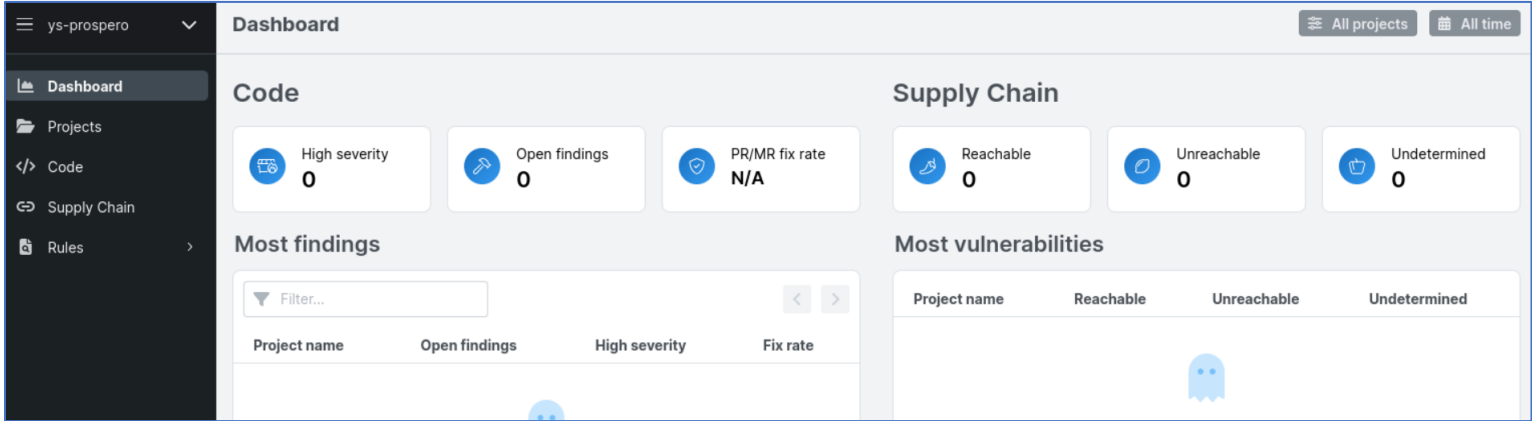
Переходим на официальную страницу по адресу <https://semgrep.dev/> и нажимаем “Start scanning for free”:



Далее логинимся с помощью своего аккаунта на GitHub и создаём название организации:



Для примера, название организации – **ys-prospero**



Теперь, для того чтобы провести локальное сканирование устанавливаем Semgrep клиент:

\$ sudo snap install semgrep

```
prospero@zorin-vm:~$ sudo snap install semgrep
[sudo] password for prospero:
2023-09-10T11:55:45+08:00 INFO Waiting for automatic snapd restart...
semgrep 1.0 from George-Andrei Iosif (iosifache) installed
prospero@zorin-vm:~$ semgrep --version
1.34.1
```

Подключаем клиент командой:

\$ semgrep login

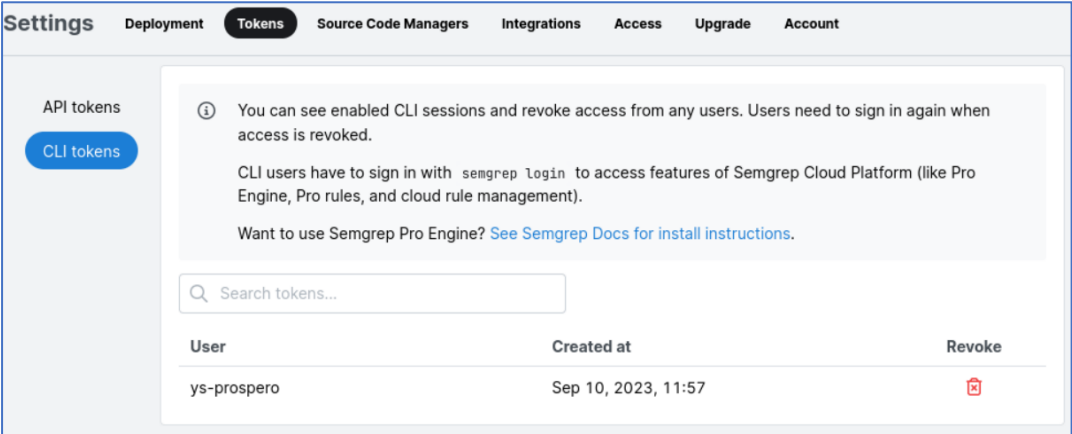
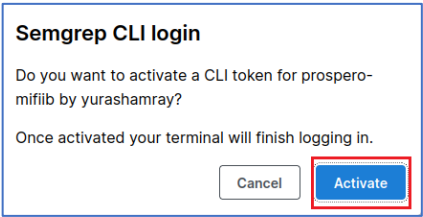
Откроется веб-страница (если автоматически на открылась, то скопируйте ссылку из терминала и вручную введите её в адресной строке браузера). Нажимаем **Activate**:

```
prospero@zorin-vm:~$ semgrep login
Login enables additional proprietary Semgrep Registry rules and running custom policies from Semgrep Cloud Platform.
Login at: https://semgrep.dev/login?cli-token=46a9a6c4-29fe-4d1b-9640-dffb798da0e&docker=False&gha=False

Once you've logged in, return here and you'll be ready to start using new Semgrep rules.
Saved login token

e048531aca3565b0be70452b55bb6b280e28daa1c385456e4d04f9092d33aac5

in /home/prosperso/snap/semgrep/5/.semgrep/settings.yml.
Note: You can always generate more tokens at https://semgrep.dev/orgs/-/settings/tokens
```



Теперь просканируем OWASP Juice Shop командой:

\$ semgrep scan --config auto ~/Downloads/juice-shop/ > scan_report.txt

```
prospero@zorin-vm:~/Downloads$ semgrep scan --config auto ~/Downloads/juice-shop/

Scan Status

Scanning 124228 files tracked by git with 1671 Code rules, 580 Pro rules:

Language      Rules  Files      Origin      Rules
-----
<multilang>    92    5317      Community   1091
json           4      915      Pro rules    580
ts             251    458
yaml           28      81
html           1       75
js             241    14
dockerfile     5       1
bash           4       1

100% 0:00:35
```

```
Scan Summary

Some files were skipped or only partially analyzed.
  Partially scanned: 29 files only partially analyzed due to parsing or internal Semgrep errors
  Scan skipped: 98 files larger than 1.0 MB, 122329 files matching .semgrepignore patterns
  For a full list of skipped files, run semgrep with the --verbose flag.

Ran 1671 rules on 1774 files: 99 findings.
```

(! для просмотра результатов статического анализа файл “scan_report.txt”)

ЗАКЛЮЧЕНИЕ

В ходе этой практической работы мы успешно установили и настроили **Burp Suite Professional**, мощный инструмент для тестирования безопасности веб-приложений. Этот инструмент предоставил нам широкие возможности для анализа трафика, обнаружения уязвимостей и проверки безопасности веб-приложений.

Также, установили **OWASP Juice Shop**, веб-приложение с открытым исходным кодом, предназначенное для обучения и практики в области кибербезопасности. Juice Shop стал нашей песочницей для тестирования и эксплуатации уязвимостей.

Продемонстрировали несколько обнаруженных уязвимостей в контексте категорий OWASP Top 10, включая уязвимости, связанные с обходом CAPTCHA, SQL-инъекциями, а также взломом учетной записи пользователя. Определили список рекомендаций по их устранению.

С помощью инструмента **Semgrep** мы провели статический анализ приложения Juice Shop на наличие потенциальных уязвимостей в коде. Это дало нам представление о том, как инструменты статического анализа могут быть использованы для обнаружения уязвимостей на этапе разработки.

Эта практическая работа подчеркивает важность безопасности веб-приложений и методов ее проверки. Мы расширили наши навыки в области кибербезопасности, обучились использованию мощных инструментов и научились обнаруживать уязвимости в рабочих веб-приложениях. Этот опыт будет ценным при работе в сфере информационной безопасности и веб-разработке.

ПРАКТИЧЕСКАЯ РАБОТА ЗАВЕРШЕНА!

MIFIIB – 10.09.2023